

Packaged Business Capabilities (PBCs)

Table of Contents

- Executive Summary
- Business Capabilities
- Domain-Driven Design
- Microservice Patterns
- Micro Frontend Patterns
- PBC-Driven Design
- The Paths to PBCs
- PBC SDLC Processes

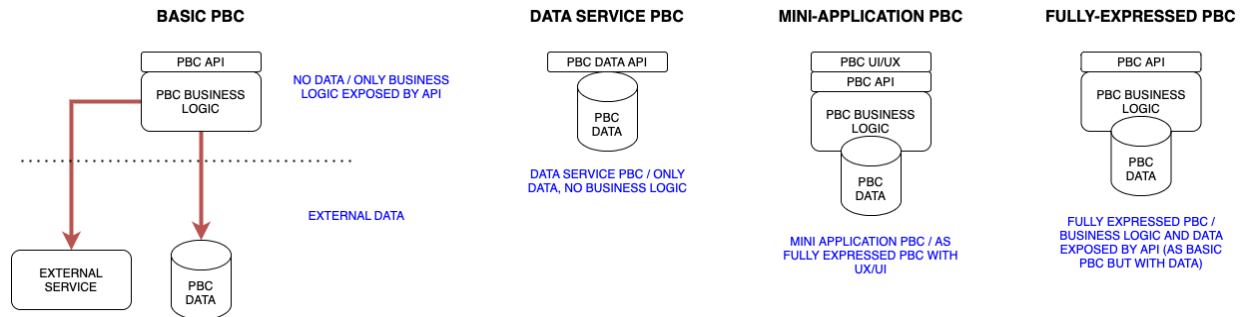
Executive Summary

Packaged business capabilities (PBCs) are encapsulated software components that each represent a well-defined business capability, recognizable as such by a business user and packaged for programmatic access. The definition does not specify the size, functional scope, or internal architecture of the implementation, but PBCs are only as useful as they are modular, discoverable, autonomous, and ready for composition (integration).

Every IT system contains multiple levels of modularization that are conceptually identical but fulfill different purposes, represent different abstraction layers, and adhere to different technological standards. The difference between a PBC and other modules is its abstraction level, which is determined by the domain of a module's objects. The business-centric development of a PBC-based platform coupled with technological abstraction allows business and technical groups to collaborate using a common system model and language.

A PBC is treated as a materialized business capability in the technical space, but it is neither a process nor a complete application. Instead, it can be viewed as a micro application dealing with only a single business capability. All PBCs fall into one of 4 architectural and functional categories:

- **Basic PBC:** Delivers a limited business service behind an API. These are widespread in modern application architecture, designed by and for IT professionals. Basic PBCs do not fully encapsulate a data entity and therefore are constrained by dependencies on external data and services.
- **Data service PBC:** Delivers analytical data. These are readily available in the form of API-centric (headless) SaaS and DaaS from vendors like Google AI and Equifax.
- **Fully-expressed PBC:** Encapsulates a business entity. These are largely autonomous and ready for assembly into new, unanticipated use cases. Their architecture is modeled on that of advanced IoT digital twins.
- **Mini-application PBC:** An encapsulated business object delivered with a user interface, ready to be deployed standalone, and collocated with similar mini-applications and/or user experiences.



Business Capabilities

The concept of a business capability provides the motivation and context for PBCs.

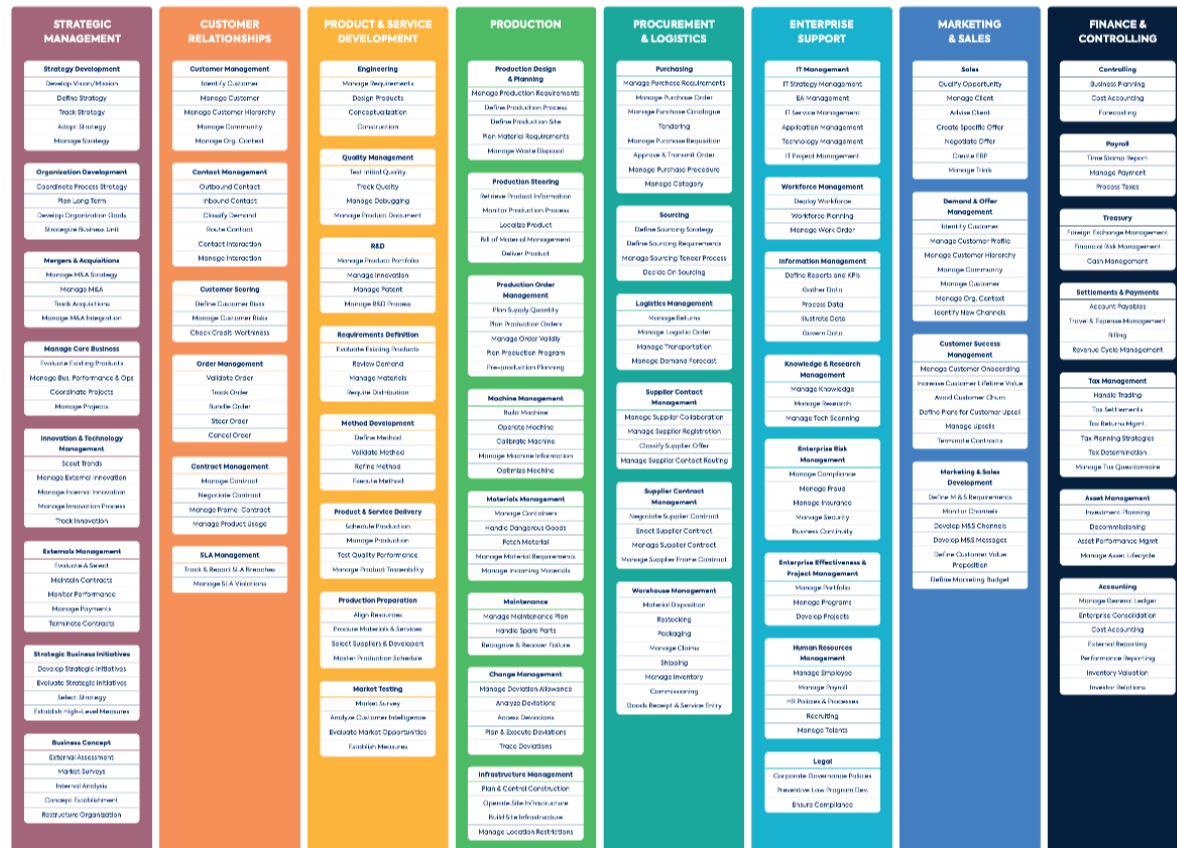
According to LeanIX¹, a **business capability** *"is the expression or the articulation of the capacity, materials, and expertise an organization needs in order to perform core functions. Enterprise Architects use business capabilities to illustrate the overarching needs of the business in order to better strategize IT solutions that meet those business needs."*

In other words, business capabilities describe what an enterprise does. Looking at the enterprise as if it were a system, its business capability map² provides a static view of the business domain. The bigger picture includes a dynamic view, which describes how business capabilities work together to produce the enterprise business value.

¹<https://www.leanix.net/en/wiki/ea/business-capability>

²<https://www.leanix.net/en/wiki/ea/business-capability#form>

Business Capability Maps



The principles governing the business map dictate that capabilities:

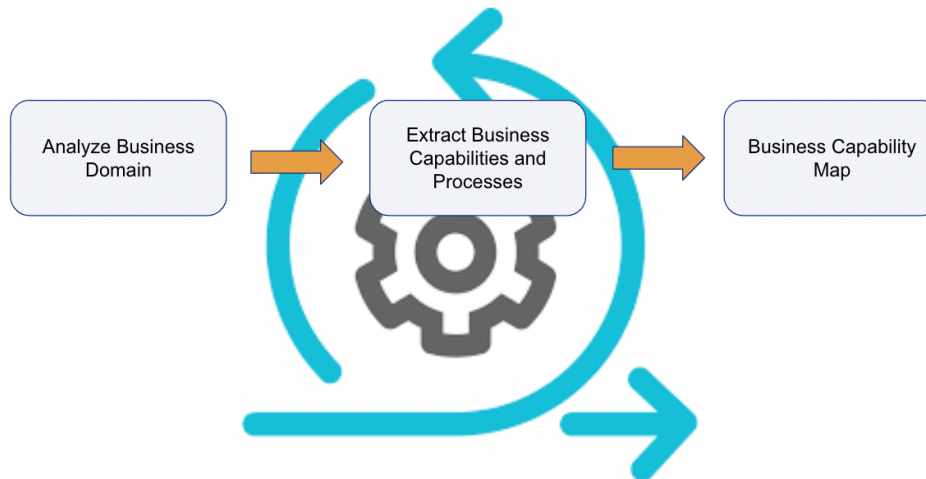
- Do not overlap
- Define what each capability is, not how it works
- Offer long-term stability; are cross-organizational
- Number 7-10 at the top level
- Are broad rather than deep
- Are accepted by business stakeholders

To understand the role and importance of business capabilities, consider how difficult it is for an enterprise to strategically drive evolution without an easy way to define its information systems. Moreover, collaboration between business and technical groups is inhibited by the lack of a common vocabulary.

The business capability model allows management to have a synthetic vision of the business, which can be treated as a system. This system is composed of a static domain (the business capabilities) that also defines the language dictionary, and a set of processes (the business dynamic view) that involve business capability collaboration and produce the enterprise business value.

At a high level, the process of defining business capabilities consists of analyzing the business domain to formulate capabilities and processes, then generating the business map.

The Process of Defining Business Capabilities



Domain-Driven Design

Domain-driven design is a software development approach that models a business domain or specific area of expertise. The model is defined through the identification of key concepts, vocabulary, and business requirements.

Domain-driven design informs software design by fostering communication and collaboration between domain experts and developers to achieve a shared understanding of business criteria and objectives. The alignment of architecture with the underlying business domain should result in language and structure that are intuitive, flexible, and maintainable. This is critical for complex systems involving multiple stakeholders and domain experts, where it is especially challenging to design software around key concepts and their interactions while accurately reflecting business requirements.

Domain-driven design has wide-ranging applications for MFE-oriented systems, including the accurate deconstruction of packaged components. It is also used extensively in the development of microservice architectures, where loosely coupled and cohesive microservices are created around business capabilities. Each microservice should have a single, well-defined purpose with an architecture that can be modified independently of others in the domain.

The two phases of domain-driven design are strategic and tactical. The strategic phase adopts a capability-centric approach to architectural design. This stage involves mapping out the

business domain, creating subsystem domain models, and constructing bounded contexts. The tactical phase establishes patterns to develop granular and precise subdomain models. Tactical patterns are designed to facilitate loosely-coupled and cohesive microservices.

The following graphic³ illustrates the flow of domain-driven design. Domain analysis prompts the definition of bounded contexts, which in turn define the tactical patterns to resolve microservices.

Domain Driven Design



1. Domain Analysis

The goal of domain analysis is a high-level definition of the final system. A domain model is created as an abstraction of the business domain. This model resolves domain criteria and constituents into an organized and clarified representation of business functions and dependencies. Core and ancillary services are differentiated into functional blocks that are abstracted from technology or implementation.

2. Bounded Contexts

A domain model can be divided into functional subsystems that rely on subsets of the information applicable to the system as a whole. By eliminating extraneous information, the representation of each subsystem can be confined to the capabilities and characteristics of that specific functional block. These subdomain models are inherently contextual and the inspiration for bounded contexts.

Bounded contexts are functional boundaries assigned to a subdomain model to differentiate between representations. Similar functionality dependent on the same sets of data and features exist within a single bounded context. Simplifying a representation to uniquely apply to a particular subsystem results in a 1:1 ratio of subsystem to bounded context.

The subsystems within bounded contexts can interact with one another despite their encapsulated functionality and parameters. To maintain the integrity of the domain model, this communication should occur through regulated and well-defined APIs.

³<https://docs.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis>